

## Unit-2

# Stack & Queue

# Unit-2

## □ Stack

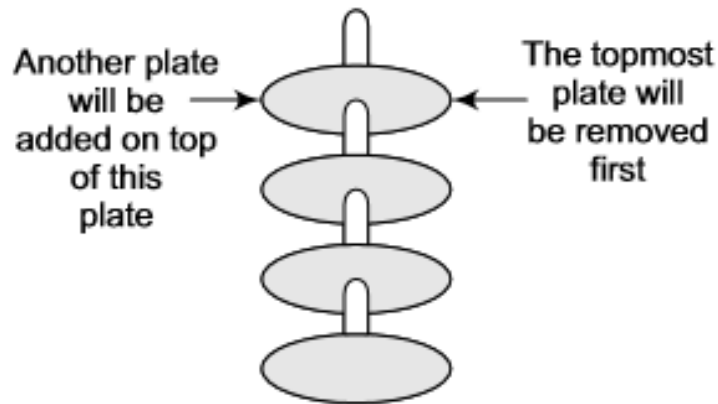
- Introduction
- Operations
- Applications of Stack
  - Recursion
  - Polish notations

## □ Queue

- Introduction
- Types of Queue
- Simple Queue and its operations
- Real life Applications of Queue

# Stack

- ❑ Stack is an important data structure which stores its elements in an ordered manner.
- ❑ Take an analogy of a pile of plates where one plate is placed on top of the other as shown in the following figure.



- ❑ A plate can be removed from the topmost position. Hence, you can add and remove the plate only at/from one position that is, the topmost position

# Stack

- ❑ ***“A stack is a linear data structure which can be implemented either using an array or a linked list”.***
- ❑ The elements in a stack are added and removed only from one end, which is called **top**.
- ❑ Hence, a stack is called a **LIFO (Last In First Out)** data structure as the element that was inserted last is the first one to be taken out.

# Array representation of Stack

- ❑ In computer memory, stacks can be represented as a linear array.
- ❑ Every stack has a variable **TOP** associated with it.
- ❑ TOP is used to store the address of topmost element of the stack.
- ❑ There is another variable named **MAX** which will be used to store the maximum number of elements that the stack can hold.
- ❑ If **TOP=NULL** means that the stack is Empty.
- ❑ If **TOP=MAX-1** means that the stack is Full.

# Array representation of Stack

- ❑ An example stack :



- ❑ The stack in above figure shows that **TOP=4**, so insertions and deletions will be done at this position.
- ❑ In this stack, five more elements can still be stored.

# Operations on a Stack

❑ A stack has four basic operations:

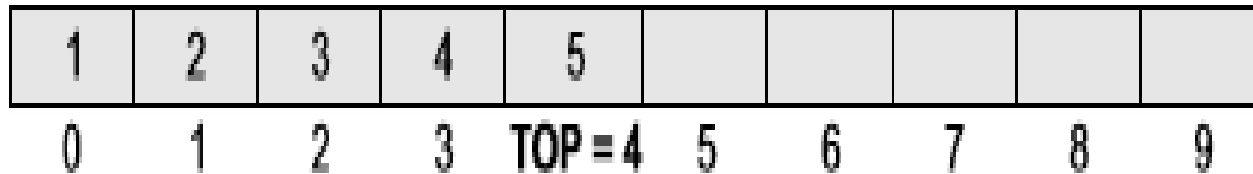
1. **Push:** adds an element to the top of the stack.
2. **Pop:** removes the element from the top of the stack.
3. **Peek:** returns the value of the topmost element of the stack.
4. **Update:** changes the value of element given by user of the stack.

# 1. Push Operation

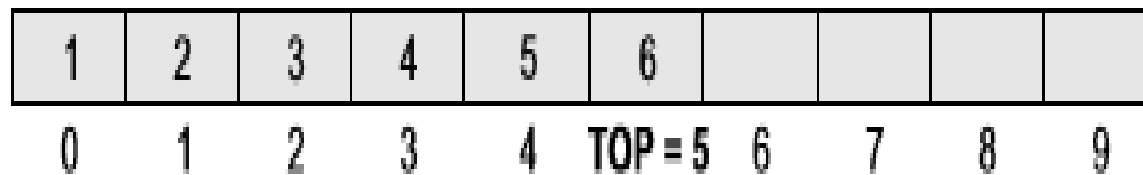
- ❑ The **PUSH** operation is used to insert an element into the stack.
- ❑ The new element is added at the topmost position of the stack.
- ❑ However before inserting the value, we must first check if **TOP=MAX-1**, as it would mean that the stack is full and no further insertions can be done on it.
- ❑ If an attempt is made to insert an element in a stack that is already full, an **OVERFLOW message** is printed.

# 1. Push Operation

- ❑ An example of stack:



- ❑ Stack after insertion:



# 1. Push Operation

- ❑ Algorithm to insert an element in a stack:

```
Step 1: IF TOP = MAX-1
        PRINT "OVERFLOW"
        Goto Step 4
    [END OF IF]
Step 2: SET TOP = TOP + 1
Step 3: SET STACK[TOP] = VALUE
Step 4: END
```

## 2. Pop Operation

- ❑ The **POP** operation is used to delete an element from the stack.
- ❑ However before deleting the value, we must check if **TOP=NULL**, as it would mean that the stack is empty and therefore no further deletions can be done on it.
- ❑ If an attempt to delete a value from a stack that is already empty is made, an **UNDERFLOW message** is printed.

## 2. Pop Operation

❑ Stack after deletion:

|   |   |   |         |   |   |   |   |   |   |
|---|---|---|---------|---|---|---|---|---|---|
| 1 | 2 | 3 | 4       |   |   |   |   |   |   |
| 0 | 1 | 2 | TOP = 3 | 4 | 5 | 6 | 7 | 8 | 9 |

❑ Algorithm to delete an element from the stack:

```
Step 1: IF TOP = NULL
        PRINT "UNDERFLOW"
        Goto Step 4
    [END OF IF]
Step 2: SET VAL = STACK[TOP]
Step 3: SET TOP = TOP - 1
Step 4: END
```

### 3. Peek Operation

- ❑ **PEEK** is an operation that returns the value of the topmost element of the stack without deleting it from the stack.
- ❑ The algorithm for the peep operation is given as follows:

```
Step 1: IF TOP = NULL  
        PRINT "STACK IS EMPTY"  
        Goto Step 3  
Step 2: RETURN STACK[TOP]  
Step 3: END
```

## 4. Update Operation

- ❑ UPDATE is an operation that updates or changes the value of specified element with specific index.
- ❑ However, the update operation first checks if the stack is empty or contains some element.

**Step 1: [ Check for Stack Underflow]**

**IF  $TOP - i + 1 = NULL$**

**PRINT "UNDERFLOW"**

**[END OF IF]**

**Step 2: [ Change  $i^{\text{th}}$  element from the top of Stack]**

**Set  $STACK[TOP-i+1] = X$**

**Step 3: END**

# Applications of Stack

- ❑ Recursion
- ❑ Polish Notations

# Recursion

- ❑ A function calls itself is called **Recursion**.
- ❑ Recursion is a repetitive process in which function calls itself repeatedly until some predefined condition is met.
- ❑ The recursive function must have a stopping condition (Termination/ Anchor Condition) or else the function would never terminate. So ultimately program **goes into infinite loop.**
- ❑ Examples where recursion has been used are:  
Factorial of a given number, Fibonacci series up to given term, etc...

# Recursion

```
#include<stdio.h>
#include<conio.h>
int factorial(int n);
void main()
{
    int n,z;
    clrscr();
    printf("\n Enter Value : ");
    scanf("%d",&n);

    z=factorial(n);
    printf("\n Factorial is : %d",z);

}
```

# Recursion

```
int factorial(int n)
```

```
{
```

```
    int f;
```

```
    if(n==1)
```

```
        return 1;
```

```
    else
```

```
        f=n*factorial(n-1);
```

```
    return f;
```

```
}
```

# Iteration Vs. Recursion

| Sr . No. | Basis for Comparison | Recursion                                                                                                                              | Iteration                                                                                                                                     |
|----------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1        | Basic                | The statement in a body of function calls the function itself.                                                                         | Allows the set of instructions to be repeatedly executed.                                                                                     |
| 2        | Format               | In recursive function, only termination condition (base case) is specified.                                                            | Iteration includes initialization, condition, execution of statement within loop and update (increments and decrements) the control variable. |
| 3        | Termination          | A conditional statement is included in the body of the function to force the function to return without recursion call being executed. | The iteration statement is repeatedly executed until a certain condition is reached.                                                          |

# Iteration Vs. Recursion

| Sr . No. | Basis for Comparison | Recursion                                                                                               | Iteration                                                                                               |
|----------|----------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 4        | Condition            | If the function does not coverage to some condition called (base case), it leads to infinite recursion. | If the control condition in the iteration statement never become false, it leads to infinite iteration. |
| 5        | Infinite Repetition  | Infinite recursion can crash the system.                                                                | Infinite loop uses CPU cycles repeatedly.                                                               |
| 6        | Applied              | Recursion is always applied to functions.                                                               | Iteration is applied to iteration statements or "loops".                                                |
| 7        | Speed                | Slow in execution.                                                                                      | Fast in execution.                                                                                      |

# Iteration Vs. Recursion

| Sr . No. | Basis for Comparison | Recursion                                                                                                  | Iteration                              |
|----------|----------------------|------------------------------------------------------------------------------------------------------------|----------------------------------------|
| 8        | Stack                | The stack is used to store the set of new local variables and parameters each time the function is called. | Does not uses stack.                   |
| 9        | Overhead             | Recursion possesses the overhead of repeated function calls.                                               | No overhead of repeated function call. |
| 10       | Size of Code         | Recursion reduces the size of the code.                                                                    | Iteration makes the code longer.       |

# Polish Notations

❑ Following are three different but equivalent notations of writing algebraic expression:

1. Infix Notation
2. Postfix Notation
3. Prefix Notation

# Infix Notation

- ❑ In the infix notation, the operator is placed between the operands.
- ❑ For example,  $A + B$ ; here, the plus operator is placed between the two operands  $A$  and  $B$ .
- ❑ Although we find it easy to write expressions using the infix notation, computers find them difficult to parse as they need a lot of information such as operator precedence, associativity rules and brackets which dictate the rules, to evaluate the expression.
- ❑ So computers work more efficiently with expressions written using prefix and postfix notations.

# Postfix Notation

- ❑ In the postfix notation, as the name suggests the operator is placed after the operands.
- ❑ For example, if an expression is written as  $A+B$  in infix notation, the same expression is written as  $AB+$  in the postfix notation.
- ❑ The order of evaluation of a postfix expression is always from left to right. Even brackets cannot alter the order of evaluation.
- ❑ Expression  $(A + B) * C$  is written in postfix notation as:

$AB+C^*$

# Postfix Notation

- ❑ Postfix operation does not even follow the rules of operator precedence.
- ❑ The operator that occurs first in the expression is operated first on the operands.
- ❑ For example, given a postfix notation  $AB+C^*$ ; while evaluation addition will be performed prior to multiplication.
- ❑ **Exercise: Convert following Infix expressions into Postfix expressions:**

$$(A - B) * (C + D)$$

# Prefix Notation

- ❑ Prefix notation is same as postfix notation but having only a difference: the operator is placed before the operands.
- ❑ For example: if  $A + B$  is an expression in the infix notation, then the corresponding expression in a prefix notation is given by:  $+AB$
- ❑ **Exercise: Convert following Infix expressions into Prefix expressions:**
  1.  $(A + B) * C$
  2.  $(A - B) * (C + D)$

## Convert Infix to Postfix Notation

- ❑ Let  $I$  be an algebraic expression written in infix notation.
- ❑  $I$  may contain parentheses, operands, and operators.
- ❑ For simplicity of the algorithm, only  $+, -, *, /$  and  $\%$  operators have been used.
- ❑ Given algorithm will transform an infix expression into postfix expression.

## Convert Infix to Postfix Notation

- ❑ The algorithm accepts an infix expression that may contain operators, operands and parentheses.
- ❑ The operators with the same precedence are performed from left-to-right.
- ❑ The algorithm uses a stack to temporarily holds operators.
- ❑ The postfix expression is obtained from left to right using the operands from the infix expression and the operators are removed from the stack.
- ❑ The very first step is to push a left parenthesis in the stack and add a corresponding right parenthesis at the end of the infix expression.
- ❑ The algorithm is repeated until the stack is empty.

Step 1: Add ")" to the end of the infix expression

Step 2: Push "(" on to the stack

Step 3: Repeat until each character in the infix notation is scanned

IF a "(" is encountered, push it on the stack

IF an operand (whether a digit or a character) is encountered, add it to the postfix expression.

IF a ")" is encountered, then

a. Repeatedly pop from stack and add it to the postfix expression until a "(" is encountered.

b. Discard the "(". That is, remove the "(" from stack and do not add it to the postfix expression

IF an operator 0 is encountered, then

a. Repeatedly pop from stack and add each operator (popped from the stack) to the postfix expression which has the same precedence or a higher precedence than 0

b. Push the operator 0 to the stack

[END OF IF]

Step 4: Repeatedly pop from the stack and add it to the postfix expression until the stack is empty

Step 5: EXIT

# Convert Infix to Postfix Notation

| Operator | PRIORITY |
|----------|----------|
| ^ , %    | 1        |
| * , /    | 2        |
| + , -    | 3        |

## Rules

- No Two Operator of same priority can stay together in the stack column.  
If Previous Operator having the higher priority then current operator then previous operator will be popped out.

# Convert Infix to Postfix Notation

**(A+B/C\* (D+E) -F)**

| Infix Character Scanned | STACK       | Postfix Expression |
|-------------------------|-------------|--------------------|
| (                       | (           |                    |
| A                       | (           | A                  |
| +                       | (+          | A                  |
| B                       | (+          | AB                 |
| /                       | (+/<br>(+/* | AB                 |
| C                       | (+/<br>(+/* | ABC                |
| *                       | (+/*        | ABC/               |
| (                       | (+*(        | ABC/               |
| D                       | (+*(        | ABC/D              |
| +                       | (+*(+       | ABC/D              |
| E                       | (+*(+       | ABC/DE             |
| )                       | (+*         | ABC/DE+            |
| -                       | (-          | ABC/DE+*+          |
| F                       | (-          | ABC/DE+*+F         |
| )                       |             | ABC/DE+*+F-        |

## Convert Infix to Postfix Notation

$$(A * B + C / (D - E))$$

| Infix Character Scanned | STACK  | Postfix Expression |
|-------------------------|--------|--------------------|
| (                       | (      |                    |
| A                       | (      | A                  |
| *                       | (*     | A                  |
| B                       | (*     | AB                 |
| +                       | (+     | AB*                |
| C                       | (+     | AB*C               |
| /                       | (+/    | AB*C               |
| (                       | (+/(   | AB*C               |
| D                       | (+/(   | AB*CD              |
| -                       | (+/( - | AB*CD              |
| E                       | (+/( - | AB*CDE             |
| )                       | (+/    | AB*CDE-            |
| )                       |        | AB*CDE-/+          |

# Convert Infix to Prefix Notation

**Step-1: Reverse the infix string. Note that while reversing the string you must interchange left and right parenthesis.**

**Step-2 : Obtain the corresponding postfix expression of the infix expression obtained as a result of Step1.**

**Step-3: Reverse the postfix expression to get the prefix expression.**

**(A\*B+C/ (D-E) )**

**( (E-D) /C+B\*A)**

| Infix Character Scanned | STACK | Postfix Expression |
|-------------------------|-------|--------------------|
| (                       | (     |                    |
| (                       | ((    |                    |
| E                       | ((    | E                  |
| -                       | ((-   | E                  |
| D                       | ((-   | ED                 |
| )                       | (     | ED-                |
| /                       | (/    | ED-                |
| C                       | (/    | ED-C               |
| +                       | (+    | ED-C/              |
| B                       | (+    | ED-C/B             |
| *                       | (+*   | ED-C/B             |
| A                       | (+*   | ED-C/BA            |
| )                       |       | ED-C/BA*+          |

**+\*AB/C-DE**

# Queue

- ❑ Like stacks, **queues are also one of the important data structures which store their elements in an ordered manner.**
- ❑ Examples :
  - People moving in an escalator.
  - People waiting for bus.
  - People standing outside the ticketing window of a cinema hall
  - Luggage kept on conveyor belt.
  - Cars lined for filling petrol.
  - Cars lined at a toll bridge.
- ❑ Same is the case with data structure.

# Queue

- ❑ A queue is a **FIFO (First In First Out)** data structure in which each element that was inserted first is the first one to be taken out.
- ❑ The elements in a queue are added at one end called the rear and removed from the other end called the front.
- ❑ Queues can be implemented using either **arrays or linked list.**

# Types of Queues

□ There are four types of Queue:

1. Simple Queue
2. Circular Queue
3. Priority Queue
4. Dequeue (Double Ended Queue)

# Simple Queue

- ❑ Queues can easily be represented using linear arrays.
- ❑ Every queue will have front and rear variables that will point to the position from where deletions and insertions can be performed respectively.

|    |   |   |    |    |    |   |   |   |   |
|----|---|---|----|----|----|---|---|---|---|
| 12 | 9 | 7 | 18 | 14 | 36 |   |   |   |   |
| 0  | 1 | 2 | 3  | 4  | 5  | 6 | 7 | 8 | 9 |

- ❑ Front : 0, rear: 5

# Operations on a simple Queue: Insertion

- ❑ Now if new value is needed to be added, say 45: then rear would be incremented by 1.

|    |   |   |    |    |    |    |   |   |   |
|----|---|---|----|----|----|----|---|---|---|
| 12 | 9 | 7 | 18 | 14 | 36 | 45 |   |   |   |
| 0  | 1 | 2 | 3  | 4  | 5  | 6  | 7 | 8 | 9 |

- ❑ Front : 0, rear: 6
- ❑ Every time a new element has to be added, the same procedure is repeated.
- ❑ An **overflow** will occur when we try to insert an element into **a queue that is already full**.
- ❑ When **rear=MAX-1**, where MAX is the size of a queue overflow occurs.
- ❑ Note that MAX-1 has been written as array index starts from 0.

# An algorithm to insert an element in a simple queue

```
Step 1: IF REAR = MAX-1
        Write OVERFLOW
        Goto step 4
    [END OF IF]
Step 2: IF FRONT = -1 and REAR = -1
        SET FRONT = REAR = 0
    ELSE
        SET REAR = REAR + 1
    [END OF IF]
Step 3: SET QUEUE[REAR] = NUM
Step 4: EXIT
```

# Operations on a simple Queue: Deletion

- ❑ Now if we want to delete an element from the queue, then the value of front will be incremented.
- ❑ Deletions are done from this one end of the queue only.

|   |   |   |    |    |    |    |   |   |   |
|---|---|---|----|----|----|----|---|---|---|
|   | 9 | 7 | 18 | 14 | 36 | 45 |   |   |   |
| 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7 | 8 | 9 |

- ❑ Front : 1, rear: 6
- ❑ An **underflow** condition occurs when we try to delete an element from **a queue that is already empty**.
- ❑ If **front=-1 and rear=-1**, this means there is no element in the queue.

## An algorithm to delete an element from a simple queue

```
Step 1: IF FRONT = -1 OR FRONT > REAR
        Write UNDERFLOW
    ELSE
        SET VAL = QUEUE[FRONT]
        SET FRONT = FRONT + 1
    [END OF IF]
Step 2: EXIT
```

# Real Life Applications of Queue

- ❑ The FIFO property of Queue makes it useful in the following kind of scenarios:
  - When a resource is shared among multiple consumers.  
Example: CPU Scheduling.
  - Ticket counter line where people who come first will get his ticket first.
  - Bank line where people who come first will done his transaction first.
  - Key press sequence in keyboard.
  - ATM booth line.

# Stack Vs. Queue

| # | STACK                                                                      | QUEUE                                                                |
|---|----------------------------------------------------------------------------|----------------------------------------------------------------------|
| 1 | Objects are inserted and removed at the same end.                          | Objects are inserted and removed from different ends.                |
| 2 | In stacks only one variable is used. It points to the top of the stack.    | In queues, two different variables are used for front and rear ends. |
| 3 | In stacks, the last inserted object is first to come out.                  | In queues, the object inserted first is first deleted.               |
| 4 | Stacks follow Last In First Out (LIFO) order.                              | Queues following First In First Out (FIFO) order.                    |
| 5 | Stack operations are called push and pop.                                  | Queue operations are called enqueue and dequeue.                     |
| 6 | Stacks are visualized as vertical collections.                             | Queues are visualized as horizontal collections.                     |
| 7 | Collection of dinner plates at a wedding reception is an example of stack. | People standing in a file to board a bus is an example of queue.     |

Thank You...!!!!